
Cascading Style Sheets

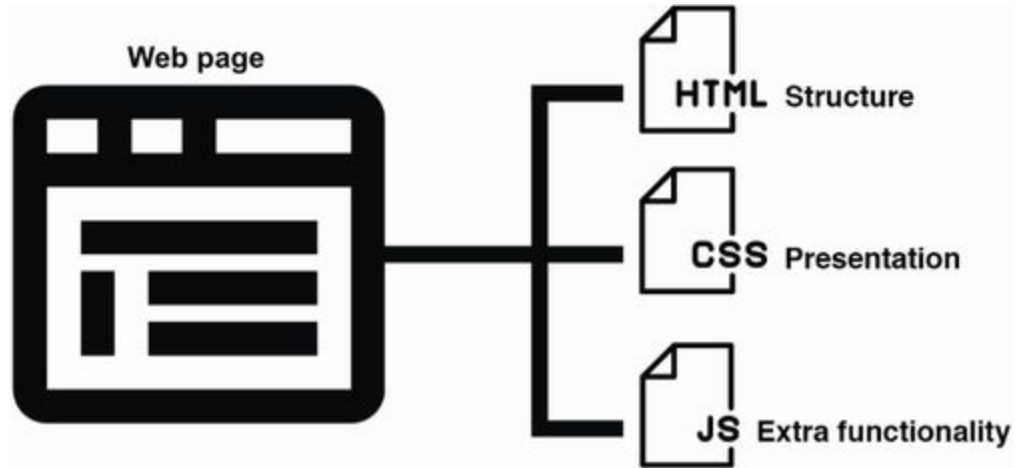
— Introduction —

Learning Outcomes

After completing this lesson, you should be able to

- Explain what Cascading Style Sheet (CSS) is
- Add CSS to HTML document in different ways
- Explain Basic CSS concepts

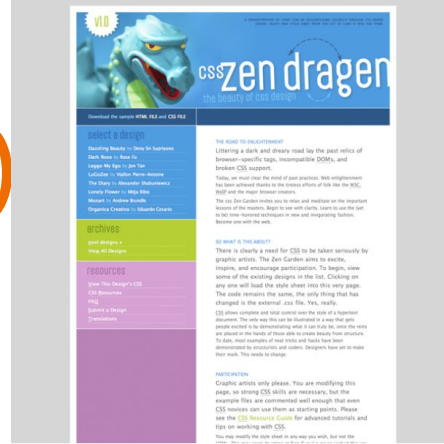
Context



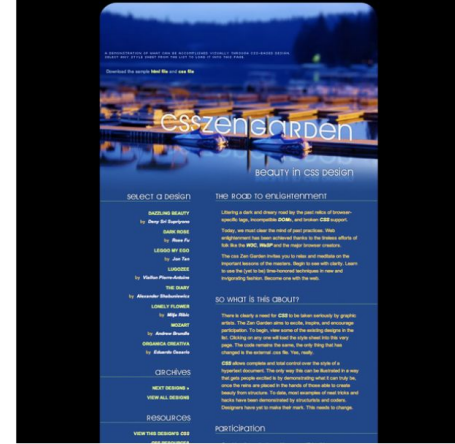
Cascading Style Sheets (CSS)

Is the W3C standard for **defining** the **presentation** of documents written in **HTML**

<https://www.csszengarden.com/>



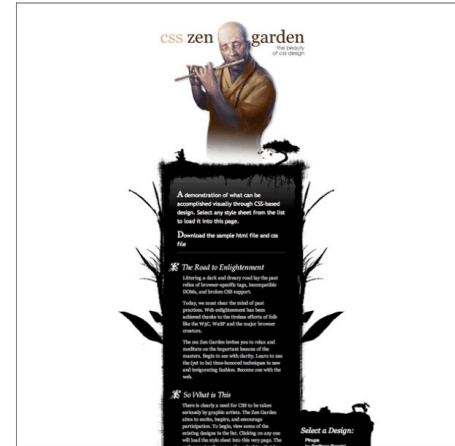
CSS Zen Garden
by Matthew Buchanan



By the Pier
by Peter Ong Kelmscott



Organica Creativa
by Eduardo Cesario



Shaolin Yokobue
by Javier Cabrera



CSS ZEN GARDEN

The Beauty of CSS Design

[VIEW ALL DESIGNS](#)

A demonstration of what can be accomplished through [CSS](#)-based design. Select any style sheet from the list to load it into this page.

Download the example [HTML FILE](#) and [CSS FILE](#)

THE ROAD TO ENLIGHTENMENT

Littering a dark and dreary road lay the past relics of browser-specific tags, incompatible [DOMs](#), broken [CSS](#) support, and abandoned browsers.

We must clear the mind of the past. Web enlightenment has been achieved thanks to the tireless efforts of folk like the [W3C](#), [WASP](#), and the major browser creators.

The CSS Zen Garden invites you to relax and meditate on the important lessons of the masters. Begin to see with clarity. Learn to use the time-honored techniques in new and invigorating fashion. Become one with the web.

SO WHAT IS THIS ABOUT?

There is a continuing need to show the power of [CSS](#). The Zen Garden aims to excite, inspire, and encourage participation. To begin, view some of the existing designs in the list. Clicking on any one will load the style sheet into this very page. The [HTML](#) remains the same, the only thing that has changed is the external [CSS](#) file. Yes, really.

[CSS](#) allows complete and total control over the style of a hypertext document. The only way this can be illustrated in a way that gets people excited is by demonstrating what it can truly be, once the reins are placed in the hands of those able to create beauty from structure. Designers and coders alike have contributed to the beauty of the web; we can always push it further.

PARTICIPATION

Strong visual design has always been our focus. You are modifying this page, so strong [CSS](#) skills are necessary too, but the example files are commented well enough that even [CSS](#) novices can use them as starting points. Please see the [CSS Resource Guide](#) for advanced tutorials and tips on working with [CSS](#).

You may modify the style sheet in any way you wish, but not the [HTML](#). This may seem daunting at first if you've never worked this way before, but follow the listed links to

MID CENTURY MODERN

by Andrew Lohman

GARMENTS

by Dan Mall

STEEL

by Steffen Knoeller

APOTHECARY

by Trent Walton

SCREEN FILLER

by Elliot Jay Stocks

FOUNTAIN KISS

by Jeremy Carlson

A ROBOT NAMED JIMMY

by [me!media](#)

VERDE MODERNA

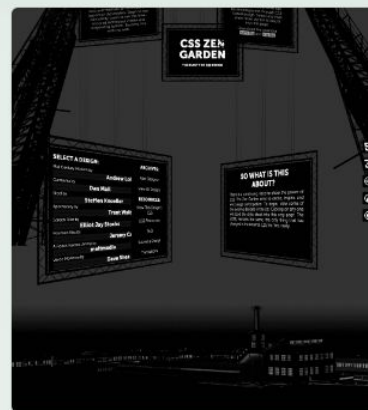
Dave Shea



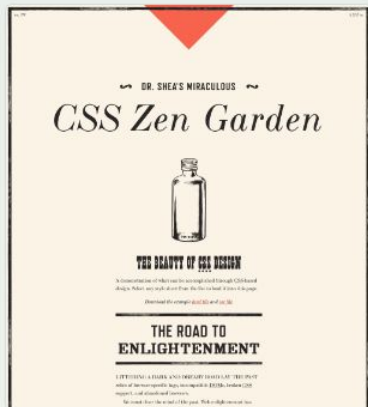
Mid Century Modern
by [Andrew Lohman](#)



Garments
by [Dan Mall](#)



Steel
by [Steffen Knoeller](#)



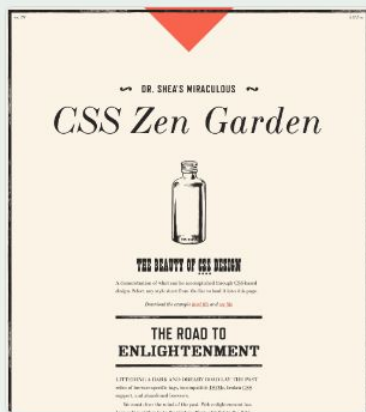
Apothecary
by [Trent Walton](#)



Screen Filler
by [Elliot Jay Stocks](#)



Fountain Kiss
by [Jeremy Carlson](#)



Apothecary
by [Trent Walton](#)



Screen Filler
by [Elliot Jay Stocks](#)



Fountain Kiss
by [Jeremy Carlson](#)



A Robot Named Jimmy
by [meltmedia](#)



Verde Moderna
by [Dave Shea](#)



Under the Sea!
by [Eric Stoltz](#)

How Stylesheets Work

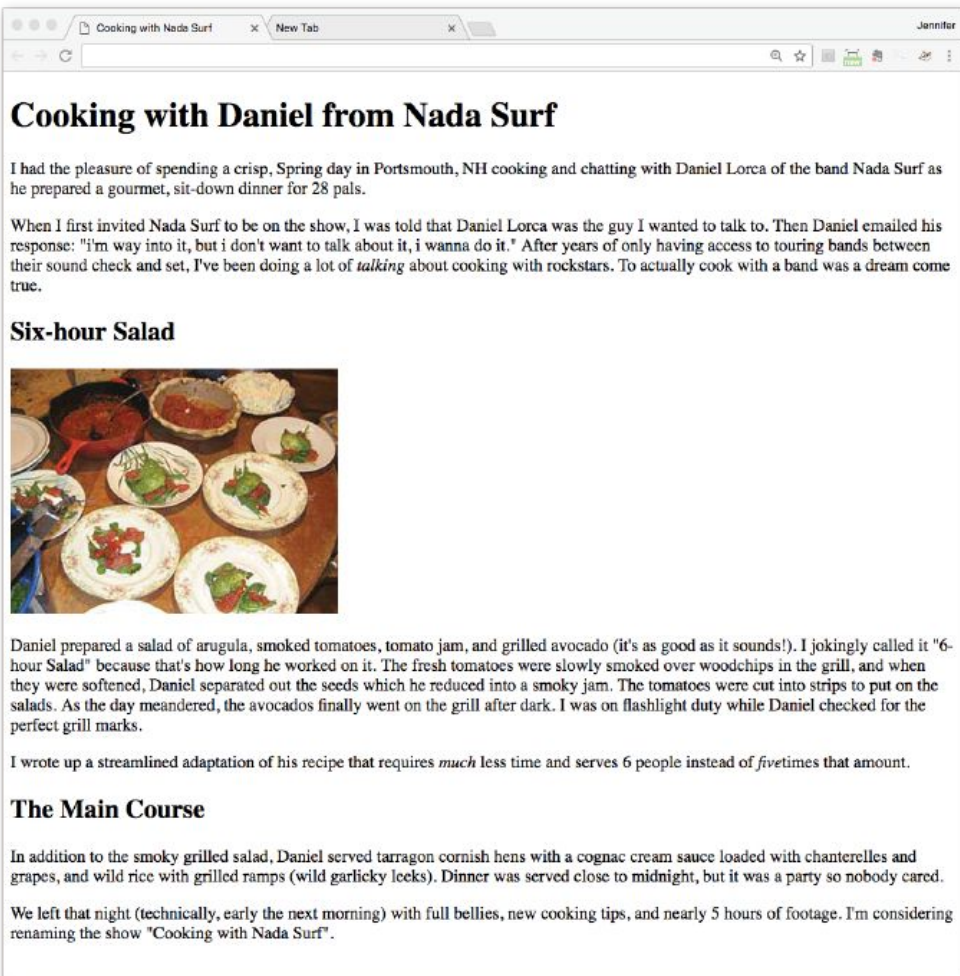
Start with a document that has been marked up in HTML

Write style rules for how you 'd like certain elements to look

Attach the style rules to the document

The first step is creating the marking up, for example as shown on the right

The first step is creating the marking up, for example as shown on the right



Writing CSS Rules

A **stylesheet** is made up of one or more style instructions (called style rules) that describe how an element or group of elements should be displayed

Example Rules

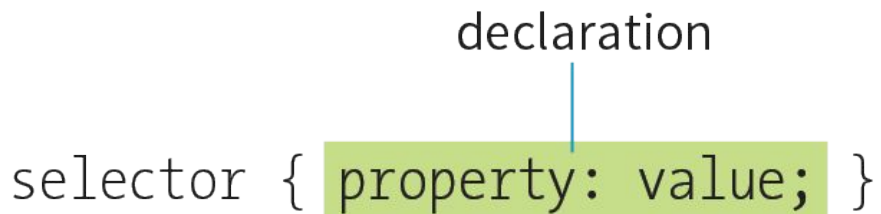
```
h1 {  
    color: green;  
}  
  
p {  
    font-size: large;  
    font-family: sans-serif;  
}
```

Writing CSS Rules

CSS Rule Syntax

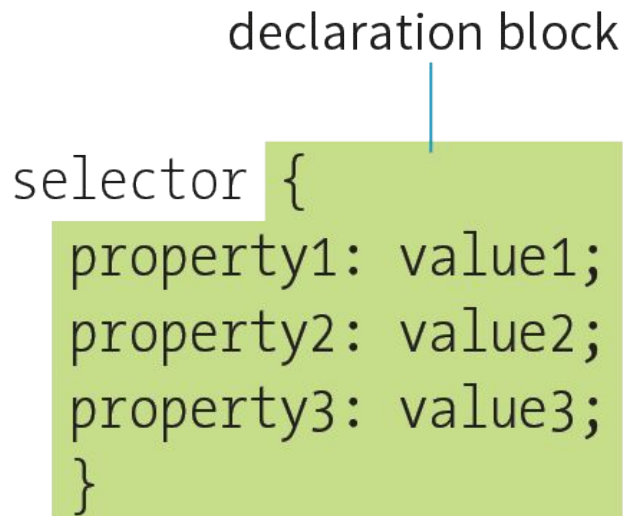
declaration

selector { property: value; }

A diagram showing a CSS rule. The text "selector { property: value; }" is displayed. A blue vertical line points from the word "declaration" above to the "property: value;" part, which is highlighted with a light green background.

declaration block

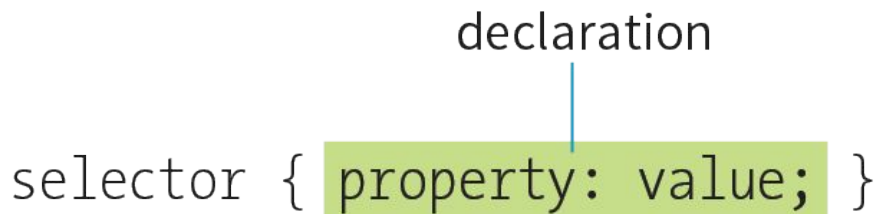
selector {
property1: value1;
property2: value2;
property3: value3;
}

A diagram showing a CSS rule with a multi-line declaration block. The text "selector {
property1: value1;
property2: value2;
property3: value3;
}" is displayed. A blue vertical line points from the word "declaration block" above to the entire block of declarations, which is highlighted with a light green background.

Writing CSS Rules

The **selector** identifies the element or elements to be affected

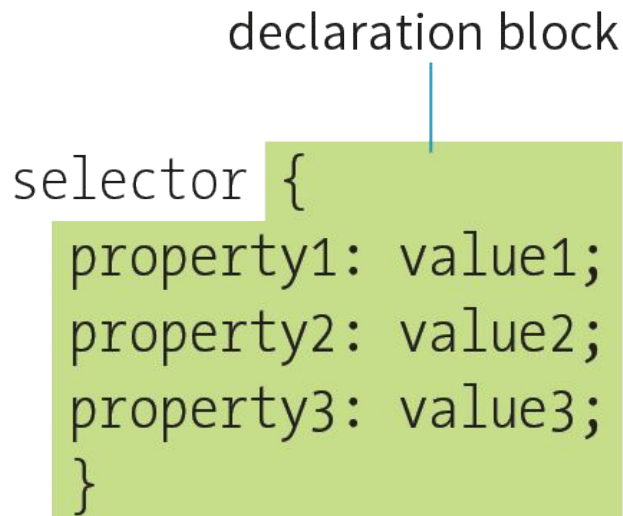
declaration



A diagram showing a CSS rule. The text "selector { property: value; }" is displayed. A blue vertical line points from the word "declaration" above to the "property: value;" part, which is highlighted with a light green background.

```
selector { property: value; }
```

declaration block



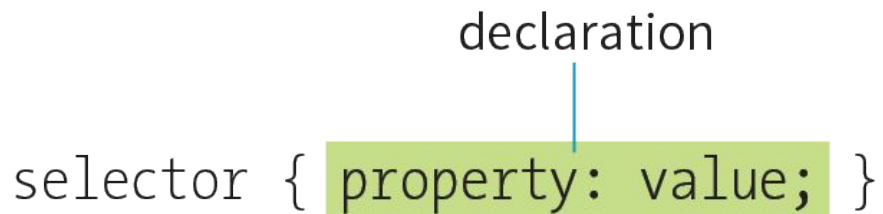
A diagram showing a CSS rule. The text "selector { property1: value1; property2: value2; property3: value3; }" is displayed. A blue vertical line points from the words "declaration block" above to the opening curly brace of the block, which is highlighted with a light green background.

```
selector {  
  property1: value1;  
  property2: value2;  
  property3: value3;  
}
```

Writing CSS Rules

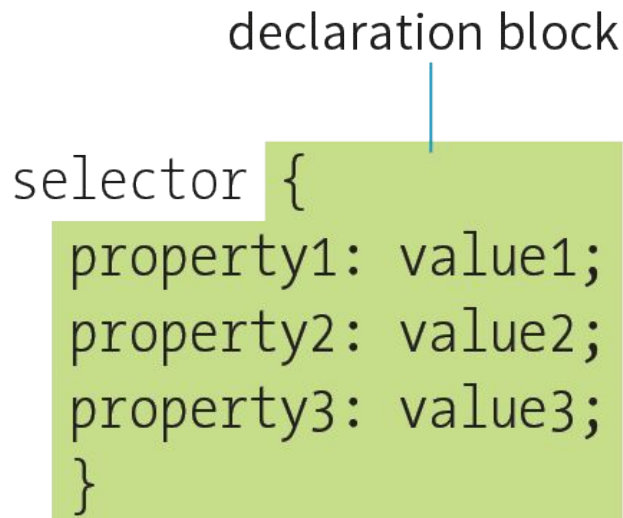
The **declaration** provides the rendering instructions

declaration



```
selector { property: value; }
```

declaration block



```
selector {  
  property1: value1;  
  property2: value2;  
  property3: value3;  
}
```

Selectors

Element type selector	Universal selector (*)
Grouped selectors	Pseudo-class selectors
Descendent selectors	Pseudo-element selectors
ID and class selectors	Attribute selectors
Child, adjacent sibling, and general sibling selectors	

Selectors

h1 and **p** are element type selectors

```
h1 {  
    color: green;  
}  
  
p {  
    font-size: large;  
    font-family: sans-serif;  
}
```

Declarations

The declaration is made up of a **property/value** pair

Each declaration must end with a semicolon to keep it separate from the following declaration

```
h1 {  
    color: green;  
}  
  
p {  
    font-size: large;  
    font-family: sans-serif;  
}
```

Grouped selectors

You can apply same style rule to multiple elements

```
p, ul, td, th { color: navy; }
```

Descendant Selectors

A descendant selector targets elements that are contained within (and therefore are descendants of) another element

Descendant selectors are indicated in a list separated by a character **space**

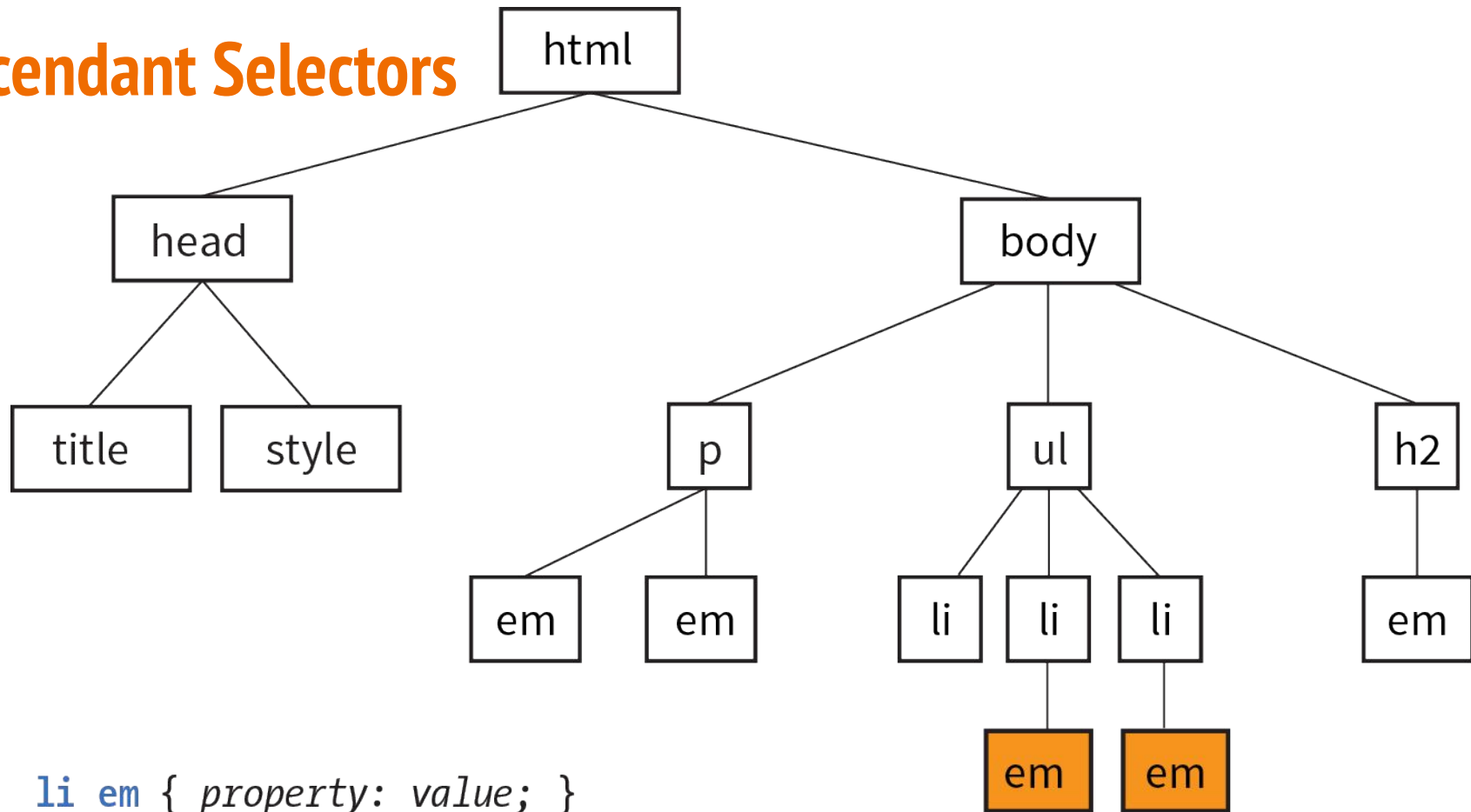
The following example targets emphasized text (**em**) elements, but only when they appear in list items (**li**)

```
li em {  
  color: olive;  
}
```

Descendant Selectors



Descendant Selectors



Child selector

A child selector is similar to a descendant selector, but it targets only the direct children of a given element

They are indicated with the greater-than symbol (>)

The following rule affects emphasized text, but only when it is directly contained in a **p** element

An em element inside a link (**a**) within the paragraph would not be affected

```
p > em {font-weight: bold;}
```

Adjacent sibling selector

Targets an element that comes directly after another element with the same parent

It is indicated with a plus (+) sign

The following rule gives special treatment to paragraphs that follow an **h1**

```
h1 + p {  
    font-style: italic;  
}
```

General sibling selector

Targets an element that comes after another element with the same parent

It is indicated with a plus (~) sign

```
h1 ~ p {  
    font-style: italic;  
}
```

The next slide shows difference between adjacent and general sibling

Header

Title of our article (heading 1)

Posted on May 16 by Lisa.

Adjacent paragraph

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Pellentesque tincidunt dapibus eleifend. Nam eu urna ipsum. Etiam consequat ac dolor et dapibus. Duis eros arcu, interdum eu volutpat ac, lacinia a tortor. Vivamus justo tortor, porttitor in arcu nec, pretium viverra ipsum. Nam sit amet nibh magna. Sed ut imperdiet orci, id finibus justo. Maecenas magna mauris, tempor nec tempor id, aliquam et nibh. Nunc elementum ut purus id eleifend. Phasellus pulvinar dui orci, sed eleifend magna ullamcorper sit amet. Proin iaculis lacus congue aliquam sodales.

1. List item 1
 - Nested item 1
 - Nested item 2
2. List item 2
3. List item 3
4. List item 4



Sibling paragraphs (but **not** adjacent)

Curabitur id augue nulla. Aliquam purus urna, aliquam eu ornare id, maximus et tellus. Aliquam eleifend sem vitae urna blandit, non bibendum tellus dignissim. Aliquam imperdiet imperdiet sapien sit amet consectetur. Nam convallis turpis felis, sed vulputate lacus eleifend a. Mauris pharetra imperdiet lacinia. Sed sit amet feugiat lectus, in consectetur magna. Vestibulum accumsan porta enim at ultricies. Vestibulum vitae massa quis massa dignissim imperdiet.

Nunc eleifend nulla lobortis porta rhoncus. Vivamus feugiat, sem vitae feugiat aliquam, orci nulla venenatis libero, vitae rhoncus nibh neque ac velit.

Etiam tempor vulputate varius. Duis at metus ut eros ultrices facilisis. Donec ut est finibus, egestas nisl eu, placerat neque. Pellentesque cursus, turpis nec sollicitudin sodales, nisi

ID Selectors

id attribute gives an element a unique identifying name (its id reference)

ID selectors allow you to target elements by their **id** values

The symbol that identifies ID selectors is the octothorpe (**#**), also known as a **hash** or pound symbol

```
<li id="sleestak">Sleestak T-shirt</li>
```

```
li#sleestak { color: olive; }
```

```
#sleestak { color: olive; }
```

Class Selectors

The **class** identifier is used to classify elements into a conceptual group

Unlike the id attribute, **multiple elements may share a class name**

An element may belong to more than one class

You can target elements belonging to the same class with a **class** selector

Class names are indicated with a period (**.**) at the beginning of the selector

Class Selectors

For example, to select all paragraphs with `class="special"`, use the following selector

```
p.special {  
    color: orange;  
}
```

Class Selectors

To apply a property to all elements of the same class, omit the element name in the selector

```
.special {  
  color: orange;  
}
```

The Universal Selector

The universal element selector (*) matches any element

The following style rule puts a 1-pixel gray border around every element in the document

```
* {  
  border: 1px solid gray;  
}
```

Pseudo-class Selectors

A pseudo-class is added to a selector to target a specific **state** of the element

Pseudo-classes are especially useful for elements that the user will **interact** with, such as **links**, **buttons**, and **form fields**

Pseudo-classes use a single colon (**:**) followed by the state of the element

Pseudo-class Selectors

```
a:link {  
  color: #1D70B8;  
}  
a:visited {  
  color: #4C2C92;  
}  
a:hover {  
  color: #003078;  
}  
a:focus {  
  outline: solid 1px crimson;  
}
```

Pseudo-element Selectors

Pseudo-elements use a double colon (::)

Pseudo-elements allows us to style a specific part of an element

```
header + p::first-letter {  
  color: crimson;  
  font-style: italic;  
}
```

Attribute value selectors

Commonly used for styling links and form elements, the attribute selector styles HTML elements that include a specified attribute

The attribute value selector looks for a specific attribute with the same value

Attribute value selectors

```
/* <a> elements with a title attribute */
```

```
a[title] {  
  color: purple;  
}
```

```
/* <a> elements with an href matching "https://example.org" */
```

```
a[href="https://example.org"]  
{  
  color: green;  
}
```

Attribute value selectors

```
/* <a> elements with an href containing "example" */
```

```
a[href*="example"] {  
    font-size: 2em;  
}
```

```
/* <a> elements with an href ending ".org", case-insensitive */
```

```
a[href$=".org" i] {  
    font-style: italic;  
}
```

Attribute value selectors

`/* <a> elements whose class attribute contains "logo" */`

```
a[class~="logo"] {  
    padding: 2px;  
}
```

Attaching the Styles to the Document

There are three ways that style information can be applied to an HTML document

Embedded stylesheets

Inline styles

External stylesheets

Embedded style sheets

The stylesheet is placed in a document via the **style** element

The rules apply only to the document where the style is embedded

```
<head>
  <title>Required document title here</title>
  <style>
    /* style rules go here */
  </style>
</head>
```

```
<!DOCTYPE html>
<html lang="en">
  <head>
    ...
    <style>
      h1, h2, h3, h4, h5, h6 {
        color: crimson;
      }
    </style>
  </head>
  <body>
    ...
  </body>
</html>
```

Inline styles

You can apply properties and values to a single element by using the style attribute in the element itself

```
<h1 style="color: red">Introduction</h1>
```

To add multiple properties, just separate them with semicolons

```
<h1 style="color: red; margin-top: 2em">Introduction</h1>
```

Inline styles

Inline styles should be avoided, unless it is absolutely necessary to override styles from an embedded or external style sheet

Inline styles are problematic in that they intersperse presentation information into the structural markup

External style sheets

A separate text-only document that contain style rules

It must be named with the `.css` suffix

This is the most preferred method for attaching stylesheets to content

```
<!DOCTYPE html>
<html>
  <head>
    <link rel="stylesheet" href="styles.css">
  </head>
  <body>
    <h1>Inline CSS</h1>
  </body>
</html>
```

Main Concepts

Inheritance

Elements Relationship

Priority

Document structure

The Box Model

Cascade

Specificity

Rule order

CSS Units

Inheritance

Styled HTML elements pass down **certain style properties** to the elements they contain

Inheritance **provides a mechanism for styling elements that don't have any explicit styles rules of their own**

Example

Let us say you have a paragraph and an `<em></em>` inside it

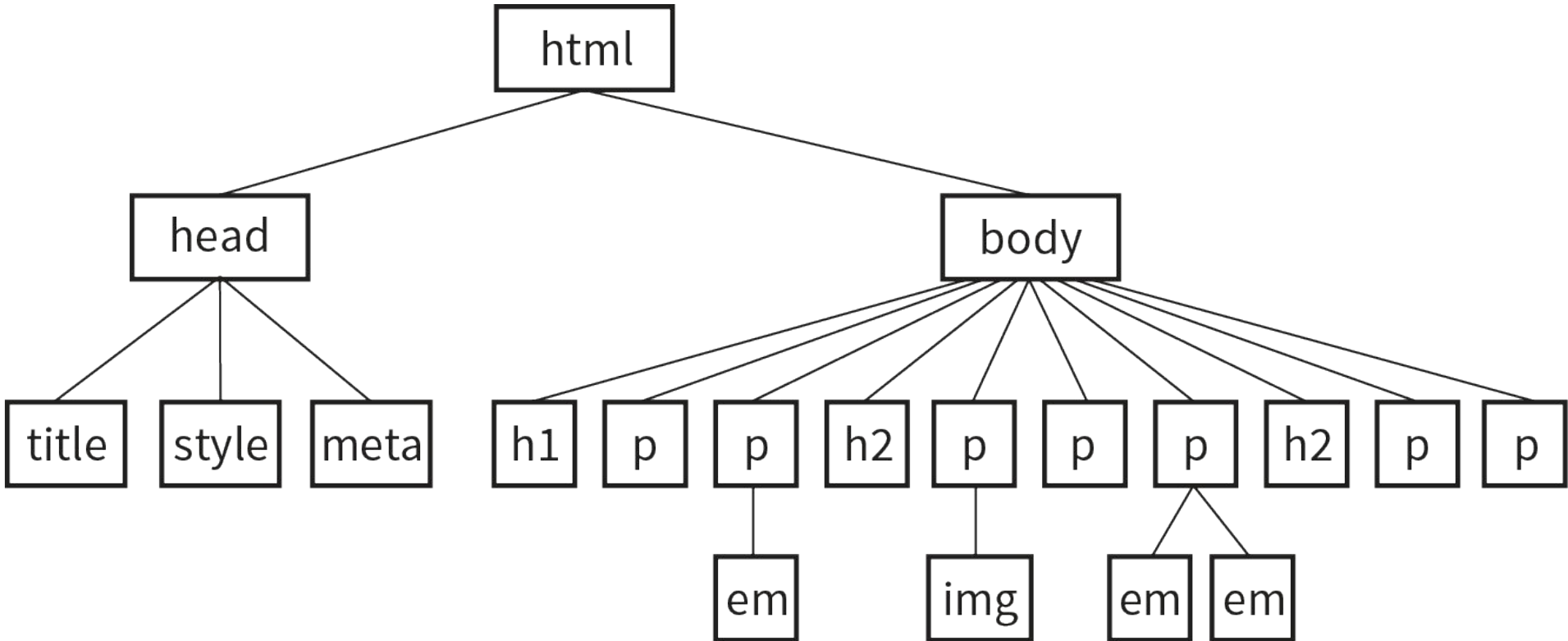
Change the font size of the paragraph and check if the font size of the content inside `<em></em>` element also changes or not

Check the inherited style rules using **Chrome DevTools**

Document structure

HTML documents have an implicit structure, or hierarchy

Document structure



Elements Relationship

All the elements contained within a given element are said to be its descendants

For example, the **h1**, **h2**, **p**, **em**, and **img** elements in the document tree shown in the previous slide are all **descendants** of the **body** element

Elements Relationship

An element that is directly contained within another element (with no intervening hierarchical levels) is said to be the **child** of that element

Conversely, the containing element is the **parent**

For example,

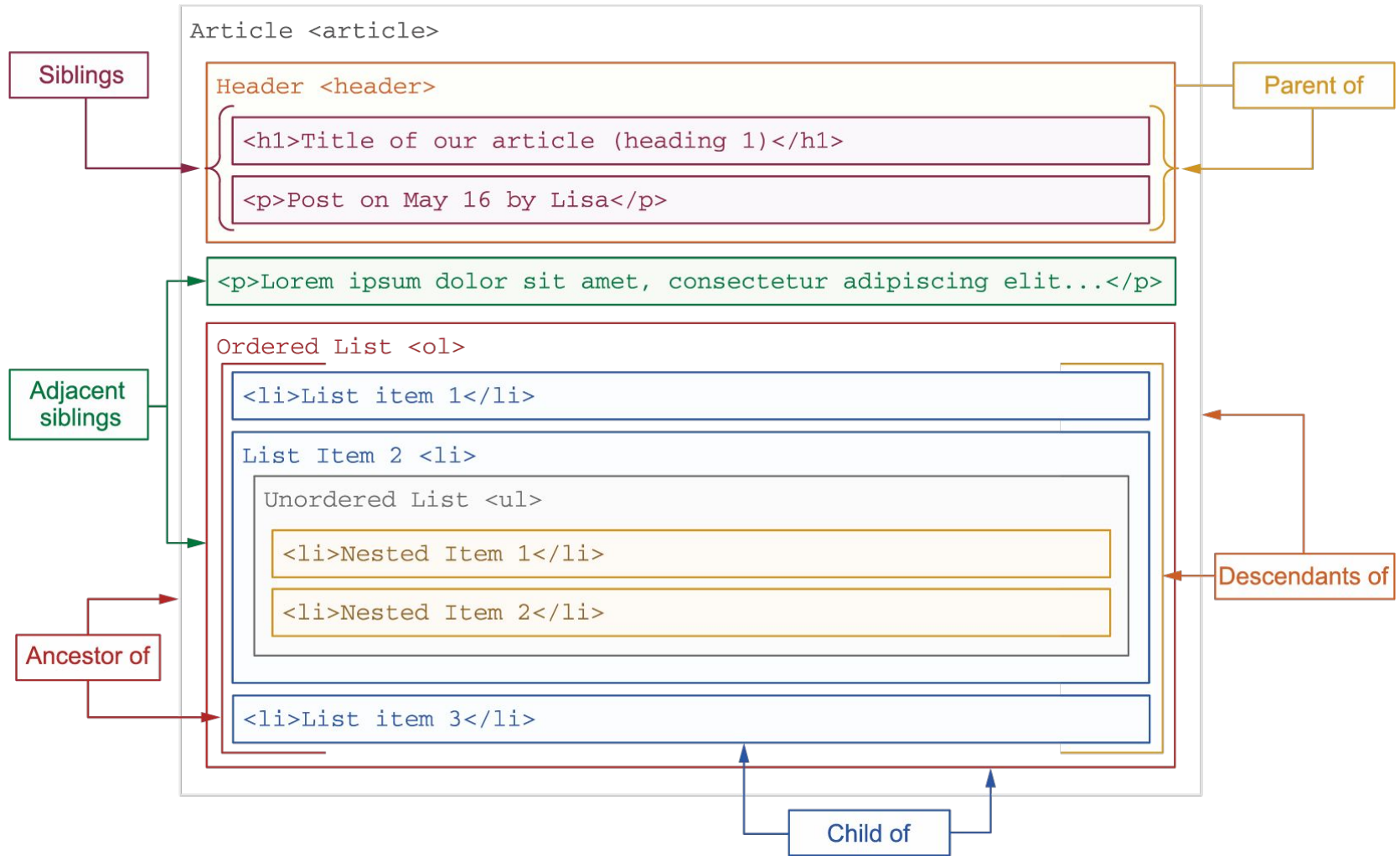
the **em** element is the **child** of the **p** element, and the **p** element is its **parent**

Elements Relationship

All of the elements higher than a particular element in the hierarchy are its **ancestors**

Two elements with the same parent are **siblings**

The following slide shows an example of different element relationships



Property Inheritance

In general, properties related to the styling of text—**font size, color, style,** and the like—are inherited by child elements

Properties such as **borders, margins, backgrounds,** and so on that affect the boxed area around the element tend not to be passed down

Example

On **Chrome DevTools** check if margin property of a paragraph is inherited by its children

Question

If you want to apply similar color to all texts in your document to which element should you add the color style rule in your stylesheet?

Cascading

CSS allows you to apply several style sheets to the same document

Cascading allows styles to overwrite or inherit from one another

The “**cascade**” refers to what happens when several sources of style information compete for control of the elements on a page

Resolving CSS rule Conflicts

When declarations conflict, the cascade considers six criteria in the following order to resolve the difference

Stylesheet origin—Where the styles come from. Your styles are applied in conjunction with the browser's default styles.

Inline styles—Whether a declaration is applied to an element via the HTML style attribute or via a CSS selector.

Layer—Styles can be defined in layers, each with a different priority

Resolving CSS rule Conflicts

When declarations conflict, the cascade considers six criteria in the following order to resolve the difference

Selector specificity—Which selectors take precedence over which

Scope proximity—Whether the styles are scoped to a portion of the Document Object Model (DOM)

Source order—Order in which styles are declared in the stylesheet

Some of these criteria are impacted by the use of the **!important** annotation

Stylesheet Origins

CSS has **three** different **stylesheet origins**

User-agent (UA) or **Browser** stylesheets

Author stylesheets

User stylesheets

Stylesheet Origins

User-agent (**browser**) stylesheets

Browsers add default styles to some elements such as **h1**

Most of the time, UA stylesheets set the **font size**, **border styles**, and some basic layout for form elements such as the text input and progress bar, which can be useful if the user stylesheet can't be found or a file-loading error occurs

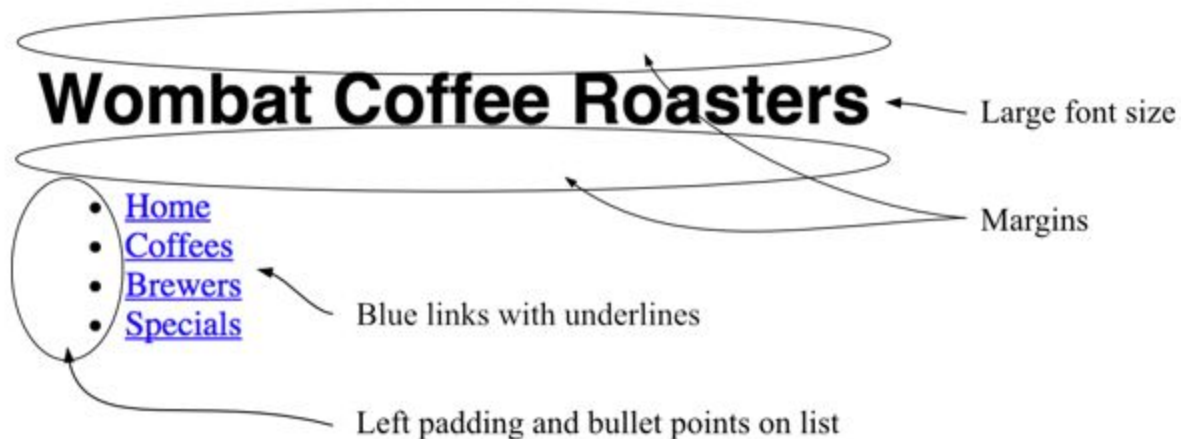
The UA stylesheet provides some fallback styling

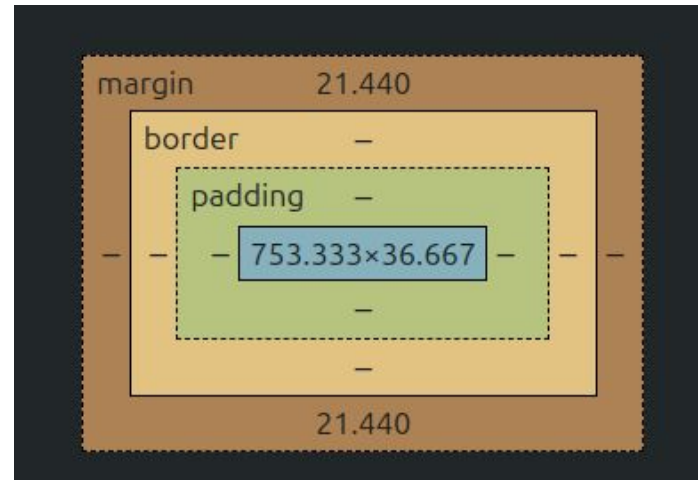
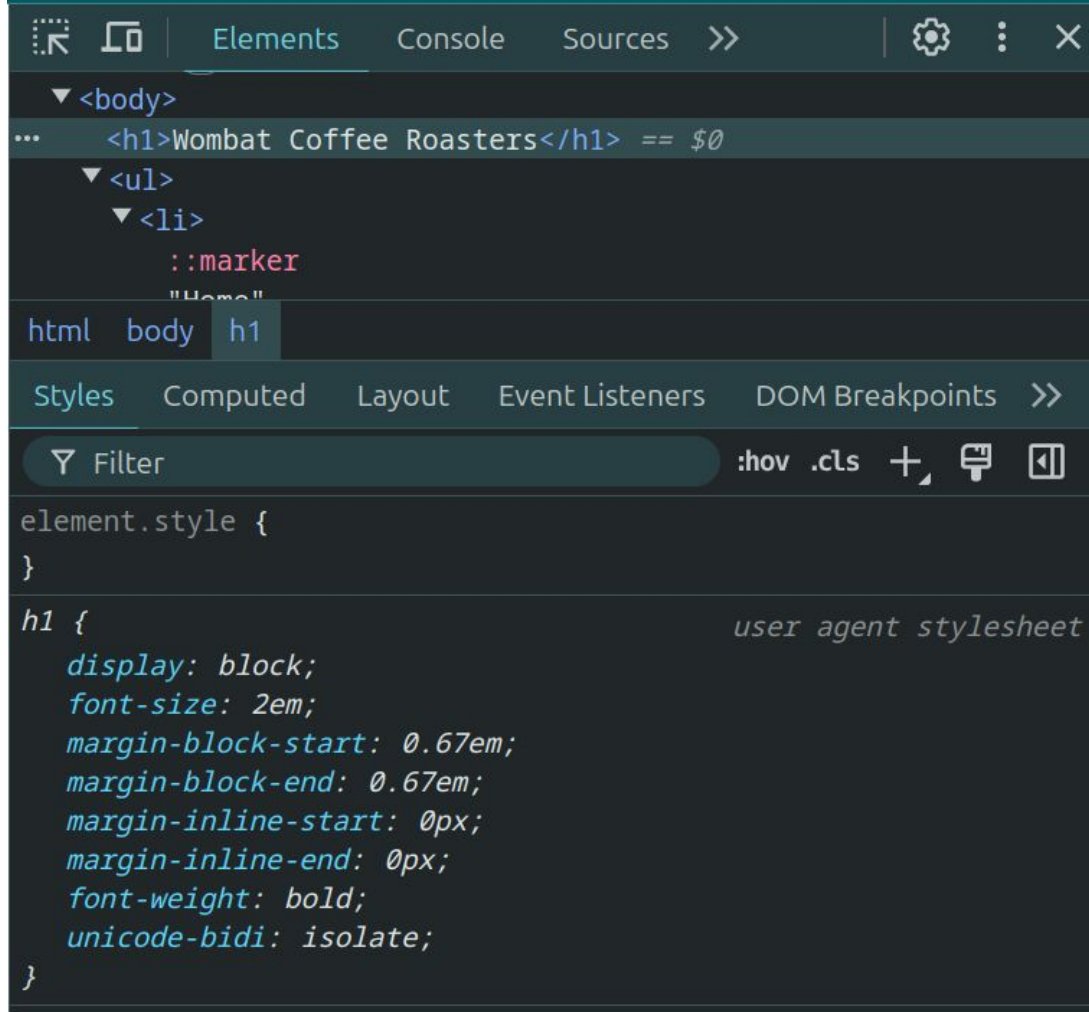
Stylesheet Origins

User-agent (browser) stylesheets

Below is shown example of UA stylesheets

The next two slides show the user agent styles





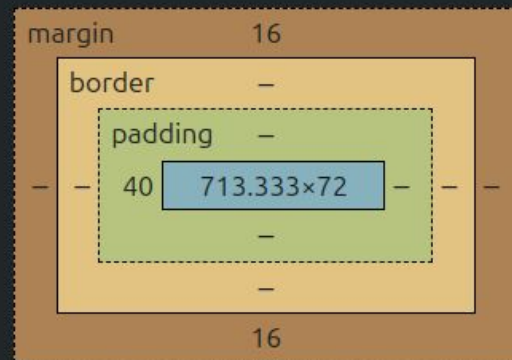
```
<!DOCTYPE html>
<html lang="en">
  <head> ... </head>
  <body>
    <h1>Wombat Coffee Roasters</h1>
    <ul> == $0
      <li> ... </li>
      <li> ... </li>
      <li> ... </li>
      <li> ... </li>
    </ul>
    <!-- Code injected by live-server -->
    <script> ... </script>
  </body>
</html>
```

Styles Computed Layout Ev

Filter

```
element.style {
}
```

```
ul {
  display: block;
  list-style-type: disc;
  margin-block-start: 1em;
  margin-block-end: 1em;
  margin-inline-start: 0px;
  margin-inline-end: 0px;
  padding-inline-start: 40px;
  unicode-bidi: isolate;
}
```



Stylesheet Origins

Author stylesheets

The stylesheets that developers write are known as author stylesheets

Stylesheet Origins

User stylesheets

A user who is accessing our web page **can use their own stylesheet** to override both author and UA styles

This option can improve their experience, especially for disabled users

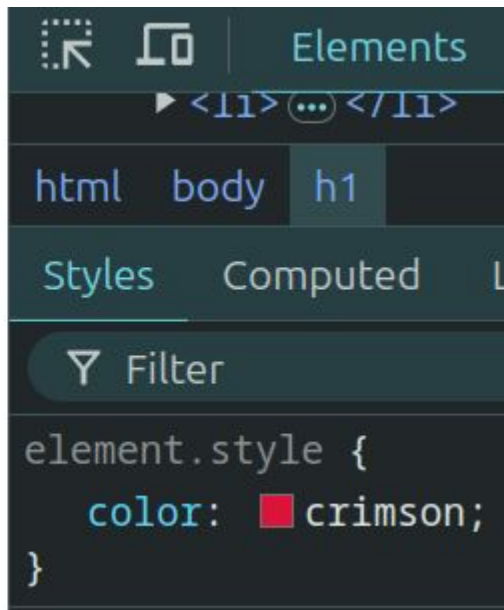
How these stylesheets are applied to the web page depends on the browser, usually through browser settings or a plugin

Stylesheet Origins

User stylesheets

Wombat Coffee Roasters

- Home
- Coffees
- Brewers
- Specials



Resolving CSS rule Conflicts

Anything explicitly defined in a rule will override inherited values

```
<body>
  <h1>Example</h1>
  <p>My paragraph</p>
</body>
```

```
body { color: red }
p { color: blue }
```

Resolving CSS rule Conflicts

Not all property values will be inherited

Theme-related styles such as color and font size will generally be inherited; layout considerations generally are not

If the **property** value **isn't** being **inherited**, the browser looks at the **type of selector** that was used and **mathematically calculates** the specificity value

Resolving CSS rule Conflicts

The browser looks at the **selector**, categorizes the **types** of selectors being used by the rule, and **applies the type value**

Then it **adds all the values** and gets a final specificity value

The biggest number wins

An example is shown in the next slide

① `button` `[type="submit"]` `#save` `span` `{ ... }`

Type selector 1 + Attribute selector 10 + ID selector 100 + Type selector 1 = 112

② `button` `[type="submit"]` `span` `{ ... }`

Type selector 1 + Attribute selector 10 + Type selector 1 = 12

Resolving CSS rule Conflicts

Specificity values by selector type are as follows

- 100** ID selectors
- 10** Class selectors, attribute selectors, and pseudo-classes
- 1** Type selectors and pseudo-elements
- 0** Universal selectors

Resolving CSS rule Conflicts

If we still have a tie, the browser looks at which **stylesheet the style originated from**

If the conflicting values come from the same stylesheet, the one later in the document wins

```
p {  
    color: blue;  
}  
  
p {  
    color: green;  
}
```

Resolving CSS rule Conflicts

If the conflicting values come from different stylesheets, the order is as follows:

User stylesheet

Author stylesheets (in the order in which they're being imported; the last one wins)

UA stylesheet

Resolving CSS rule Conflicts

The `!important` annotation

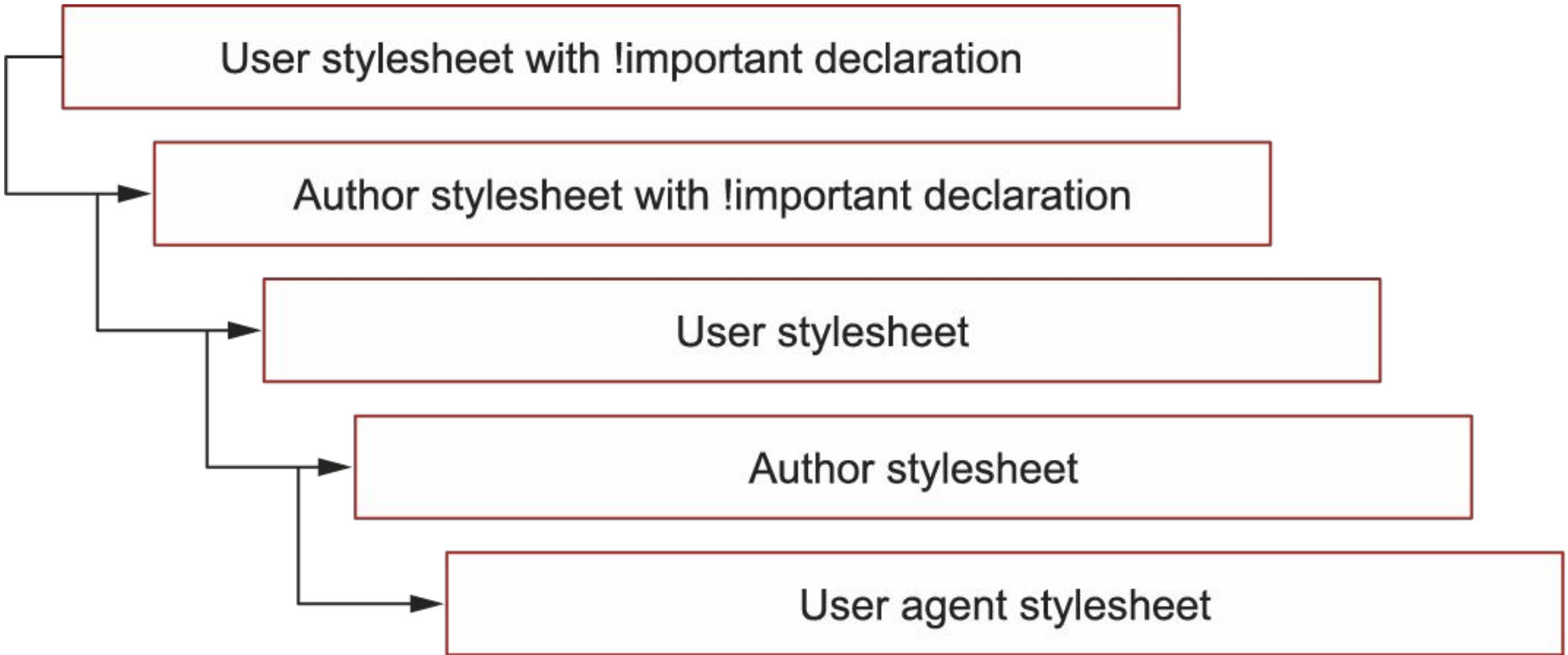
It overrides the specificity

When a value has the `!important` annotation the value with the annotation automatically wins

```
p {  
    color: blue !important;  
}
```

Resolving CSS rule Conflicts

If multiple values have the **!important** annotation, the browser follows the normal flow shown in the next slide



```
graph TD; A[User stylesheet with !important declaration] --> B[Author stylesheet with !important declaration]; B --> C[User stylesheet]; C --> D[Author stylesheet]; D --> E[User agent stylesheet];
```

User stylesheet with !important declaration

Author stylesheet with !important declaration

User stylesheet

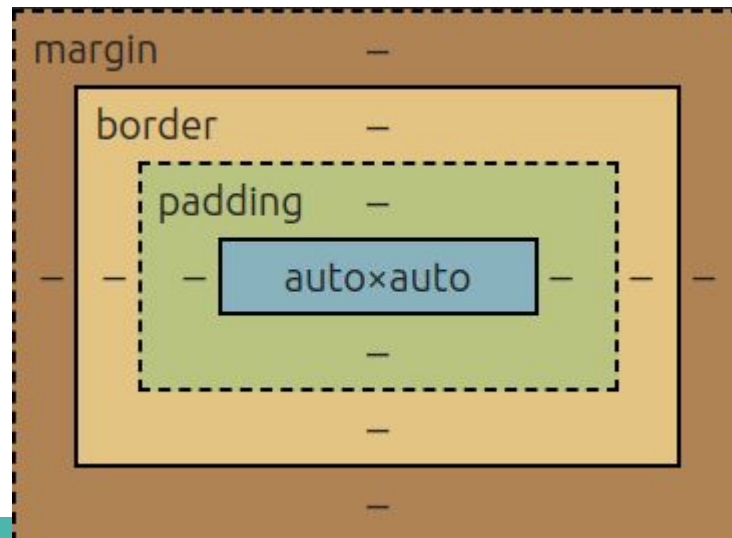
Author stylesheet

User agent stylesheet

The Box Model

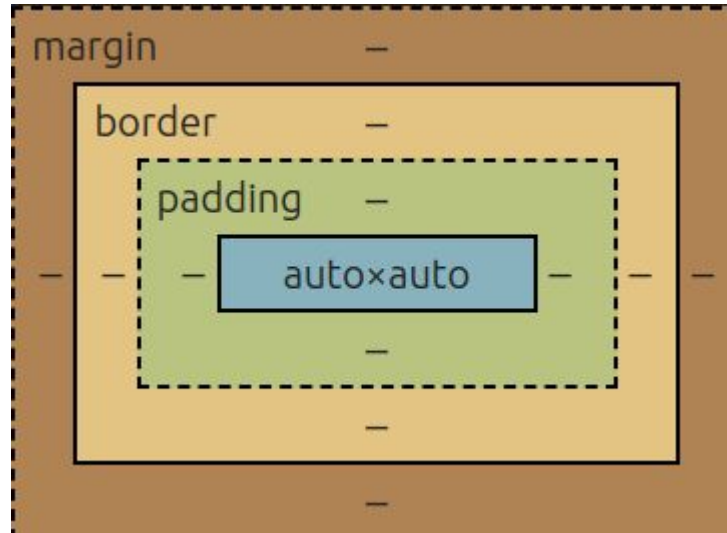
Browsers see every element on the page (both **block** and **inline**) as being contained in a little **rectangular box**

You can apply properties such as **borders**, **margins**, **padding**, and **backgrounds** to these boxes



Example

Use **Chrome DevTools** to check the rectangular box around each element



CSS Units

CSS3 provides a variety of units of measurement

They fall into two broad categories: **absolute** and **relative**

CSS Units

Absolute units have predefined meanings or real-world equivalents

With the exception of pixels, they are not appropriate for web pages that appear on screens

Relative units are **based on the size of something else**, such as the default text size or the size of the parent element

Child elements do not inherit the relative values of their parent, but rather the resulting calculated value

Absolute Units

px

pixel, defined as equal to 1/96 of an inch in CSS3

in

inches

mm

millimeters

Relative units

em

a unit of measurement equal to the current font size

ex

x-height, approximately the height of a lowercase “x” in the font

rem

root em, equal to the em size of the root element (**html**)

Relative units

ch

zero width, equal to the width of a zero (0) in the current font and size

vw

viewport width unit, equal to 1/100 of the current viewport (browser window) width

vh

viewport height unit, equal to 1/100 of the current viewport height

Relative units

vmin

viewport **min**imum unit, equal to the value of **vw** or **vh**, whichever is smaller

vmax

viewport **max**imum unit, equal to the value of **vw** or **vh**, whichever is larger

CSS Units: Percentages

Percentages are another common measurement value for web page elements

Percentages are calculated relative to another value, such as the value of a property applied to the current element or its parent or ancestor

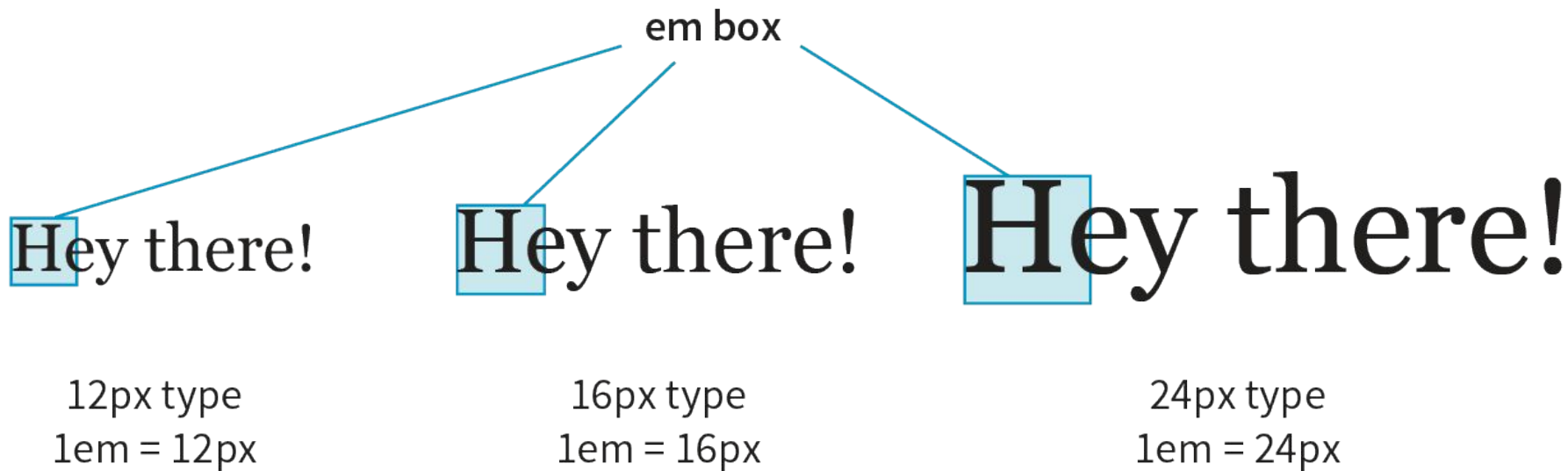
When used for page layouts, percentage values ensure that page elements stay proportional

The **rem** unit

In modern browsers, the default root font size is **16 pixels**; therefore, a rem is equivalent to a 16-pixel unit

The **em** unit

For text with a font size of **16 pixels**, an **em** measures **16 pixels**; for **12-pixel** text, an **em** equals **12 pixels**



The `em` unit

□□ **This is a 24pt Heading**

□□ **A Heading in 20pt**

□□ Lorem ipsum dolor sit amet, consectetur adipiscing elit. Aliquam facilisis imperdiet pretium. Proin fermentum urna sed arcu efficitur tincidunt. Donec id libero euismod, venenatis augue in, vestibulum lectus. Donec ultricies finibus eleifend. Aenean egestas augue sem, vitae ultricies libero fringilla a. Aliquam at tellus purus. Donec accumsan metus sit amet leo volutpat pellentesque.

```
h1, h2, p { margin-left: 2em; }
```

Viewport percentage lengths (vw/vh)

The viewport width (**vw**) and viewport height (**vh**) units are relative to the size of the viewport (browser window)

A **vw** is equal to **1/100** the width of the viewport

A **vh** is equal to **1/100** the height of the viewport

Viewport-based units are useful for making images and text elements stay the full width or height of the viewport

Viewport percentage lengths (vw/vh)

```
header {  
  width: 100vw;  
  height: 100vh;  
}
```

```
img {  
  width: 50vw;  
  height: 50vh;  
}
```

Assignment Overview

Create Profile and Portfolio website

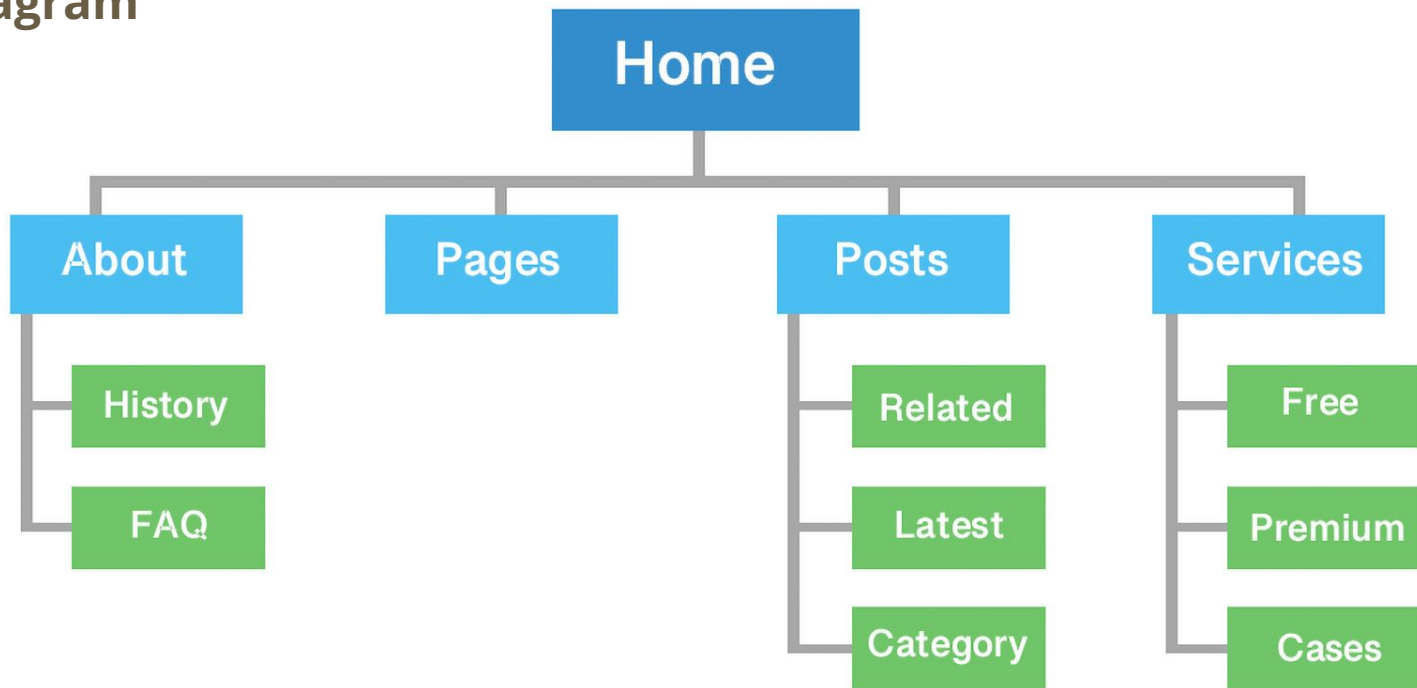
About yourself, hobby, education, CV, interest, adventures...

At least 5 pages

Should have `p`, `h1` ... `h5`, `main`, `article`, `footer`, `header`,
`section`, `nav`, ..., `table`, `ol/ul`, `img/picture`, `video`, `audio`,
`figure`, ...

Assignment Overview

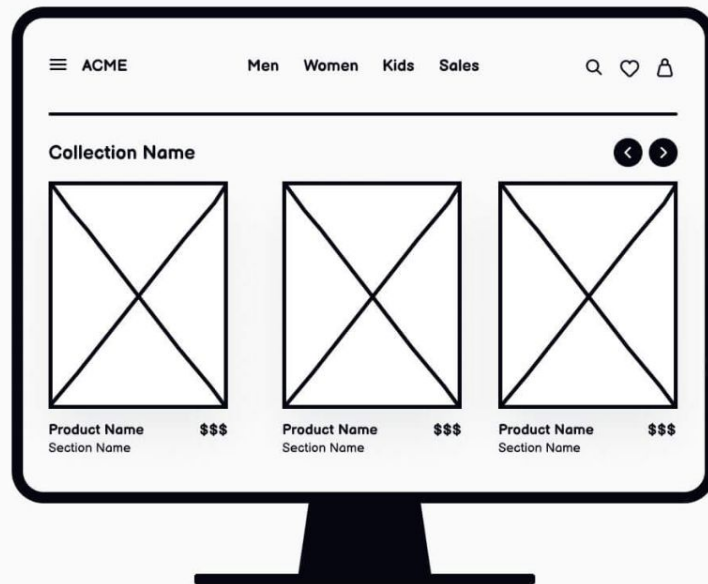
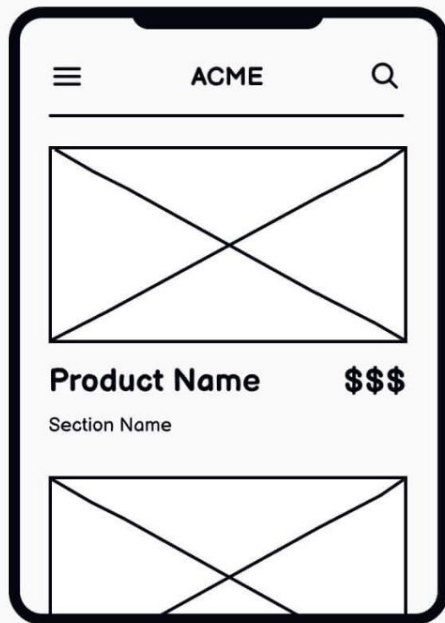
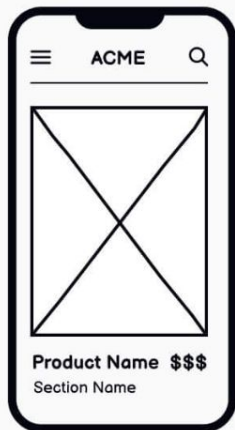
Site Diagram



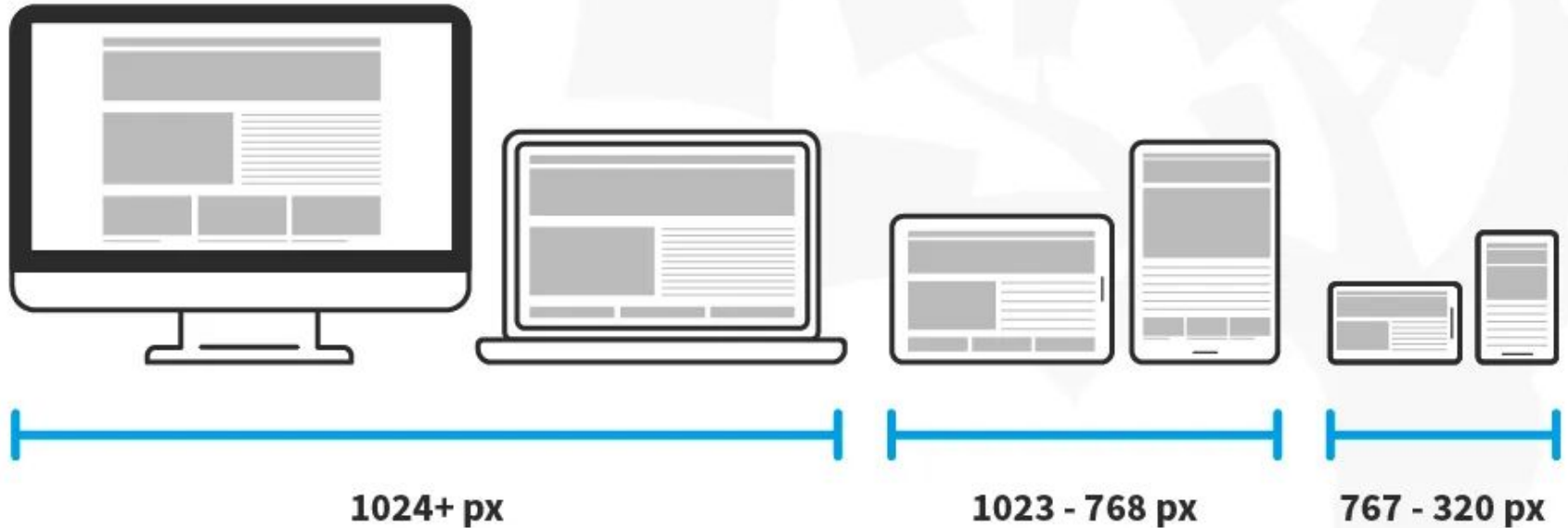
Assignment Overview

Create Wireframe by hand (pen and pencil) take a clean picture using your mobile phone or scan it

For **mobile (portrait and landscape)**, **tablet (portrait and landscape)**, and **desktop**



Responsive Design



Screen Sizes



Reference

<https://careerfoundry.com/en/blog/ux-design/what-is-a-wireframe-guide/>

<https://webstyleguide.com/wsg3/3-information-architecture/4-presenting-information.html>

<https://www.smashingmagazine.com/2016/11/wireframe-perfectionist-guide/>